



SAPIENZA
UNIVERSITÀ DI ROMA

“SAPIENZA” UNIVERSITY OF ROME

FACULTY OF INFORMATION ENGINEERING,
COMPUTER SCIENCE AND STATISTICS

DEPARTMENT OF COMPUTER, CONTROL
AND MANAGEMENT ENGINEERING

MASTER'S DEGREE IN ARTIFICIAL INTELLIGENCE AND ROBOTICS

Elective in Robotics

Lecture notes

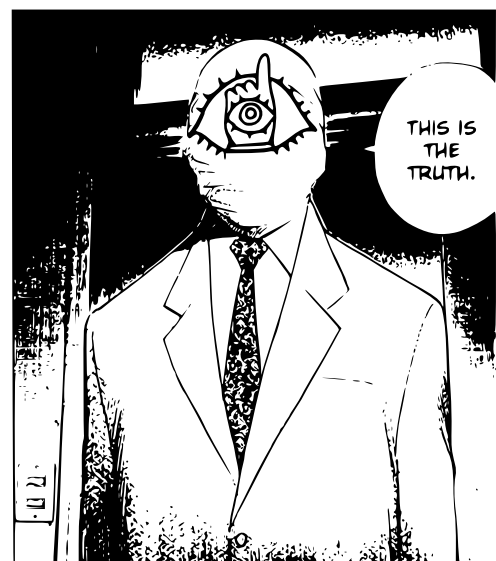
Author

Marco Casu

<https://github.com/CasuFrost>

October 6, 2026

This document summarizes and presents the topics for the Elective in Robotics course for the Master's degree in Artificial Intelligence and Robotics at Sapienza University of Rome. The document is free for any use. If the reader notices any typos, they are kindly requested to report them to the author.



CONTENTS

I	Modeling and Control of Multi-Rotor UAVs	3
1	Introduction	4
1.1	Quadrotor Modeling	5
1.2	Quadrotor Control	9
1.2.1	Linear Control	10
1.2.2	Basic Control and Planning Problems	10
2	Control Methods	13
2.1	Backstepping	13
2.1.1	Backstepping for Height Control	15
2.1.2	PD Compared With Backstepping	16
2.1.3	Integral Backstepping	16
2.2	Dynamic Feedback Linearization	17
2.2.1	Static Linearization	17
2.2.2	Input-Output Linearization	18
2.2.3	Linearization of the Quadrotor Model	19

Part I

**Modeling and Control of Multi-Rotor
UAVs**

CHAPTER

1

INTRODUCTION

UAV stands for **Unmanned Aerial Vehicle**, also known by other names depending on the domain, like *RPVs (Remotely Piloted Vehicles)* in the military domain, or *Drones* in common parlance. We will use all of the three names interchangeably. The methods presented are valid also for remotely operated or autonomous vehicles (at different levels of autonomy).

The term UAVs refers to a broad category of vehicles, in general, any aerial vehicle without a pilot on board is a UAV. We will examine UAVs composed of multiple rotors, in particular **quadrotors**.



Figure 1.1: Design-based classification.

The quadrotors are used in various situations:

- inspection, maintenance,
- communication networks,
- search and rescue operations,
- aerial transportation, manipulation.

These activities require a vertical, stationary and slow flight. A quadrotor is characterized by a high maneuverability, can remain suspended in the air without moving (hovering), and perform a *vertical take-off and landing (VTOL)*, unlike other vehicles like airplanes, that require a horizontal movement in order to take-off and land.

1.1 Quadrotor Modeling

We call VTOLs the vehicles that can perform a vertical take-off and landing, these are modeled with the North-East-Down (NED) reference frame:

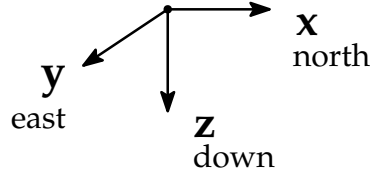


Figure 1.2: The NED reference frame.

This frame, often called SR_i , has the z axis pointing down to the center of the Earth. According to the dynamics of rigid bodies given by the Newton-Euler formalism, the translational dynamics of a body in the SR_i frame is given by

$$\sum_j \mathbf{F}_j = m\dot{\mathbf{V}} \quad (1.1)$$

where $\mathbf{F}_j$ are the external forces applied to the center of mass, and $\dot{\mathbf{V}}$ is the acceleration of the center of mass expressed in SR_i . The rotational dynamics is given by

$$\sum_j \mathbf{M}_j = J\dot{\boldsymbol{\Omega}} + \boldsymbol{\Omega} \times J\boldsymbol{\Omega} \quad (1.2)$$

where $\mathbf{M}_j$ are the external moments acting on the center of mass, $\boldsymbol{\Omega}$ is the angular velocity of the body expressed in the body frame, denoted SR_b and J is the inertia tensor, also expressed in SR_b .

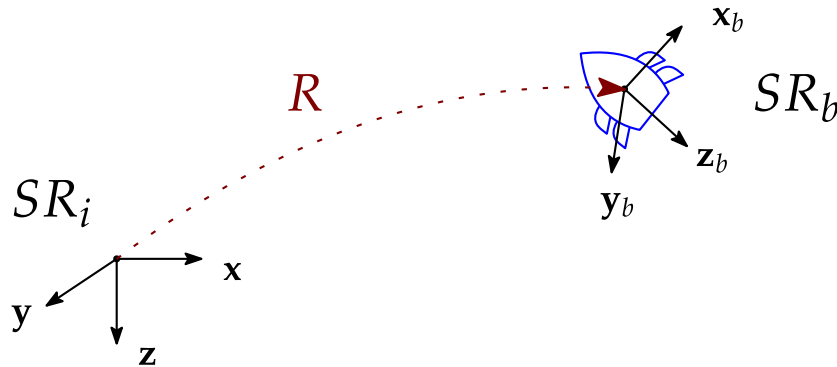


Figure 1.3: SR_b is the reference frame attached to the center of mass of the vehicle, R is the rotation matrix between the two frames.

A more explicit expression of the forces in Equation (1.1) is the following:

$$m\dot{\mathbf{V}} = -TR\mathbf{z} + \mathbf{F}_e(\mathbf{V}, \dot{\mathbf{V}}, \boldsymbol{\Omega}, \dot{\boldsymbol{\Omega}}, \mathbf{d}(t)) + R\Sigma_R\boldsymbol{\tau} \quad (1.3)$$

where:

- $\mathbf{d}(t)$ are the external disturbances that do not depend on the vehicle's state.
- $\mathbf{F}_e$ are the external forces, that depend on the state.

- T is the **thrust**, that depends on the number and allocation of the rotors, while $-\mathbf{z}$ is the direction pointing upward (according to the NED frame as in Figure 1.2). R is the rotation matrix that describes the orientation of our vehicle with respect to the fixed NED frame SR_i ,
- hence the product $-TR\mathbf{z}$ is the main upward thrust.
- $\boldsymbol{\tau}$ is the torque generated by the propellers.
- $R\Sigma_R\boldsymbol{\tau}$ are the parasitic forces generated by the rotors/actuators. The coupling matrix Σ_R maps the torques/rotations of the motors into small translational lateral forces (then rotated into SR_i via R).

We know that the derivative of the rotation matrix R depends on the skew-symmetric matrix of the angular velocity:

$$\dot{R} = R \cdot S(\boldsymbol{\Omega}) \quad (1.4)$$

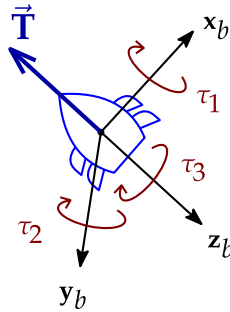
$S(\boldsymbol{\Omega})$ is important since it appears in the explicit expression of the term $J\dot{\boldsymbol{\Omega}}$ in Equation (1.2):

$$J\dot{\boldsymbol{\Omega}} = -S(\boldsymbol{\Omega})J\boldsymbol{\Omega} + \boldsymbol{\tau} + \boldsymbol{\tau}_e(\mathbf{V}, \dot{\mathbf{V}}, \boldsymbol{\Omega}, \dot{\boldsymbol{\Omega}}, \mathbf{d}(t)) + \boldsymbol{\tau}_g + \Sigma_T T \mathbf{z} \quad (1.5)$$

where:

- $\boldsymbol{\tau}_e$ are the torques induced by the external forces.
- $\Sigma_T T \mathbf{z}$ is a parasitic moment. Σ_T is a matrix that represents the geometric misalignment between the effective axis of the total thrust T and the center of mass of the vehicle. If the thrust vector does not pass exactly through the center of gravity, it generates a parasitic torque proportional to the thrust itself.
- $\boldsymbol{\tau}_g$ is the internal gyroscopic moment due to the inertia of the propellers rotating at high speed (particularly relevant in quadcopters when they change attitude rapidly).

We denote $\vec{T} = -T\mathbf{z}_b$ the push in the opposite direction of $\mathbf{z}_b$ expressed in the vehicle frame SR_b .



A quadrotor is a UAV with four motors located at the extremities of a cross-shaped frame (Figure 1.4) and is controlled by imposing a desired angular speed on each rotor.

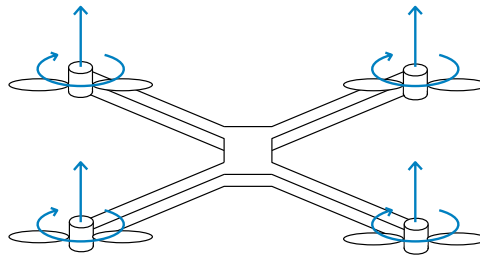


Figure 1.4: Schematic of a quadrotor.

Let's identify a disc on each rotor, that follows the profile of the propellers: we denote f_i the scalar force on each propeller, which is perpendicular to the disc, its value is:

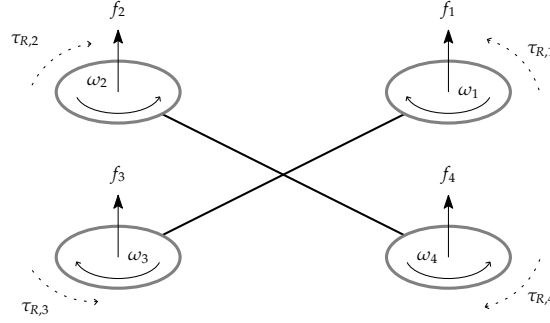
$$f_i = b\omega_i^2 \quad (1.6)$$

where ω_i is the angular speed of the i -th propeller, and b is a constant *thrust factor*, that depends on the rotor geometry and profile, and on air density. The rotational speed of each blade is controlled by a low level controller.

We denote $\tau_{R,i}$ the drag torque generated by the i -th rotor as it spins in the air:

$$\tau_{R,i} = d\omega_i^2 \quad (1.7)$$

where d is a *drag factor* that, as the thrust factor, depends on the rotor.



We denote φ, ϑ, ψ the roll-pitch-yaw angles of the vehicle with respect to the fixed reference frame SR_i , hence the already mentioned rotation matrix is:

$${}^iR_b = \begin{pmatrix} \cos \psi \cos \vartheta & \cos \psi \sin \vartheta \sin \varphi - \sin \psi \cos \varphi & \cos \psi \sin \vartheta \cos \varphi + \sin \psi \sin \varphi \\ \sin \psi \cos \vartheta & \sin \psi \sin \vartheta \sin \varphi + \cos \psi \cos \varphi & \sin \psi \sin \vartheta \cos \varphi - \cos \psi \sin \varphi \\ -\sin \vartheta & \cos \vartheta \sin \varphi & \cos \vartheta \cos \varphi \end{pmatrix}. \quad (1.8)$$

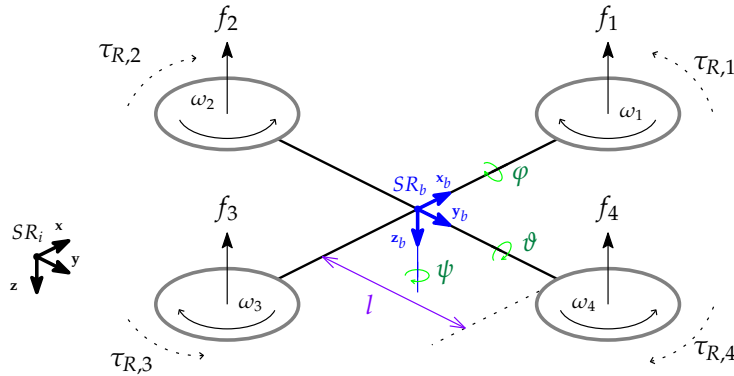


Figure 1.5: Configuration of the quadrotor.

The linear velocity $\mathbf{V}$ of our quadrotor is expressed in SR_i while the angular velocity $\mathbf{\Omega}$ is expressed in SR_b , we can make the reference system explicit using superscripts as follows:

$${}^i\mathbf{V} = \begin{pmatrix} v_x \\ v_y \\ v_z \end{pmatrix}, \quad {}^b\mathbf{\Omega} = \begin{pmatrix} p \\ q \\ r \end{pmatrix}. \quad (1.9)$$

Let $\omega_1, \omega_2, \omega_3, \omega_4$ be our control inputs (the velocity of the rotors), f_i and $\tau_{R,i}$ are derived from these quantities as in Equations (1.6) and (1.7). The total thrust will be:

$$T = f_1 + f_2 + f_3 + f_4. \quad (1.10)$$

We recall that on the quadrotor, a force $\vec{T} = -T\mathbf{z}_b$ acts in the same direction of $\mathbf{z}_b$ but with the opposite sense. Intuitively, if the forces f_i are identical, we can expect the quadrotor to accelerate towards $-\mathbf{z}_b$ without rotating on itself. The presence of these forces, if they are not identical, will cause a rotation of

the body. The torques generated on the axes $\mathbf{x}_b, \mathbf{y}_b, \mathbf{z}_b$ are the following:

$$\tau_\varphi = l(f_2 - f_4), \quad (1.11)$$

$$\tau_\vartheta = l(f_1 - f_3), \quad (1.12)$$

$$\tau_\psi = -\tau_{R,1} + \tau_{R,2} - \tau_{R,3} + \tau_{R,4}. \quad (1.13)$$

l represents the arm length (or lever arm) of the quadrotor, as shown in Figure 1.5.

For simplicity, we can assume that the inertia matrix shown in Equation (1.2) is diagonal:

$$J = \begin{pmatrix} I_x & 0 & 0 \\ 0 & I_y & 0 \\ 0 & 0 & I_z \end{pmatrix} \quad (1.14)$$

the angular velocity instead can be expressed in function of the roll-pitch-yaw angles:

$$\boldsymbol{\Omega} = \begin{pmatrix} p \\ q \\ r \end{pmatrix} = \begin{pmatrix} 1 & 0 & -\sin \vartheta \\ 0 & \cos \varphi & \cos \vartheta \sin \varphi \\ 0 & -\sin \varphi & \cos \vartheta \cos \varphi \end{pmatrix} \begin{pmatrix} \dot{\varphi} \\ \dot{\vartheta} \\ \dot{\psi} \end{pmatrix}. \quad (1.15)$$

Let's see in details what are the applied forces and moments. We explicit the sum of forces of the translational dynamics (that we recall is expressed in the fixed frame SR_i):

$$\sum_j \mathbf{F}_j = \underbrace{\begin{pmatrix} 0 \\ 0 \\ mg \end{pmatrix}}_{\text{weight}} + \underbrace{{}^i R_b \begin{pmatrix} 0 \\ 0 \\ -T \end{pmatrix}}_{\text{actuation}} + \mathbf{F}_A + \mathbf{F}_D \quad (1.16)$$

where $\mathbf{F}_A$ are the aerodynamics related forces, which mainly include the aerodynamic resistance of the body and arms of the aircraft to advancement, as well as any parasitic lift effects generated by the shape of the drone or by the rotor discs in translation, while $\mathbf{F}_D$ are the disturbances (for example, wind).

The sum of moments is the following:

$$\sum_j \mathbf{M}_j = \underbrace{\begin{pmatrix} L_\varphi \\ L_\vartheta \\ L_\psi \end{pmatrix}}_{\text{gyroscopic effects}} + \underbrace{\begin{pmatrix} \tau_\varphi \\ \tau_\vartheta \\ \tau_\psi \end{pmatrix}}_{\text{actuation}} + \boldsymbol{\tau}_A + \boldsymbol{\tau}_D. \quad (1.17)$$

$\boldsymbol{\tau}_A$ is the resulting torque induced by the aerodynamic forces distributed when they do not have an application point coinciding with the center of gravity of the vehicle, combined with the rotational drag that the air opposes to the rotations of the rigid body on roll, pitch and yaw. $\boldsymbol{\tau}_D$ is the moment produced by the disturbances.

Definition 1.1.1 : State of the Quadrotor

Given the previous premises, we define the state of a quadrotor as a vector in $\mathbb{R}^{12}$ defined as follows:

$$\boldsymbol{\xi} = (x, y, z, v_x, v_y, v_z, \varphi, \vartheta, \psi, p, q, r)^T \quad (1.18)$$

here x, y, z is the position of the center of mass of the quadrotor expressed in SR_i , v_x, v_y, v_z its velocity, φ, ϑ, ψ the roll-pitch-yaw angles between SR_i and SR_b , and p, q, r the angular velocity of the quadrotor expressed in SR_b .

The evolution in time of the quadrotor state is given by a dynamical system of the following form:

$$\dot{\boldsymbol{\xi}} = f(\boldsymbol{\xi}) + g(\boldsymbol{\xi})\mathbf{u} \quad (1.19)$$

where $\mathbf{u}$ are the inputs of the system defined as follows:

$$\mathbf{u} = \begin{pmatrix} T \\ \tau_\varphi \\ \tau_\vartheta \\ \tau_\psi \end{pmatrix} \quad (1.20)$$

these quantities depend on $f_i, \tau_{R,i}$, which in turn depend on the angular velocity of the rotors ω_i , which we assume we can control perfectly.

Definition 1.1.2 : Time Evolution of the Quadrotor

Let Ω_r be the average blades rotation velocity, and I_r the blades inertia, the explicit expression of the system in Equation (1.19) is the following:

$$\dot{x} = v_x \quad (1.21)$$

$$\dot{y} = v_y \quad (1.22)$$

$$\dot{z} = v_z \quad (1.23)$$

$$\dot{v}_x = F_{A,x} - (\cos(\psi) \sin(\vartheta) \cos(\varphi) + \sin(\psi) \sin(\varphi)) \frac{T}{m} \quad (1.24)$$

$$\dot{v}_y = F_{A,y} - (\sin(\psi) \sin(\vartheta) \cos(\varphi) - \sin(\varphi) \cos(\psi)) \frac{T}{m} \quad (1.25)$$

$$\dot{v}_z = F_{A,z} + g - \cos(\vartheta) \cos(\varphi) \frac{T}{m} \quad (1.26)$$

$$\dot{\varphi} = p + \sin(\varphi) \tan(\vartheta) q + \cos(\varphi) \tan(\vartheta) r \quad (1.27)$$

$$\dot{\vartheta} = \cos(\varphi) q - \sin(\varphi) r \quad (1.28)$$

$$\dot{\psi} = \sin(\varphi) \sec(\vartheta) q + \cos(\varphi) \sec(\vartheta) r \quad (1.29)$$

$$\dot{p} = \tau_{A,x} + \frac{I_r}{I_x} q \Omega_r + \frac{I_y - I_z}{I_x} q r + \frac{\tau_\varphi}{I_x} \quad (1.30)$$

$$\dot{q} = \tau_{A,y} + \frac{I_r}{I_y} p \Omega_r + \frac{I_z - I_x}{I_y} p r + \frac{\tau_\vartheta}{I_y} \quad (1.31)$$

$$\dot{r} = \tau_{A,z} + \frac{I_x - I_y}{I_z} p q + \frac{\tau_\psi}{I_z}. \quad (1.32)$$

1.2 Quadrotor Control

The model in Definition 1.1.2 is too complex for control purposes. We neglect aerodynamics and gyroscopic effects, and we assume small changes for φ and ϑ such that

$$\begin{pmatrix} \dot{\varphi} & \dot{\vartheta} & \dot{\psi} \end{pmatrix}^T \simeq \begin{pmatrix} p & q & r \end{pmatrix}^T \quad (1.33)$$

we assume that the shape is symmetric and we neglect the disturbances.

Definition 1.2.1 : Simplified Time Evolution of the Quadrotor

$$\ddot{x} = -(\cos(\psi) \sin(\vartheta) \cos(\varphi) + \sin(\psi) \sin(\varphi)) \frac{T}{m} \quad (1.34)$$

$$\ddot{y} = -(\sin(\psi) \sin(\vartheta) \cos(\varphi) - \sin(\varphi) \cos(\psi)) \frac{T}{m} \quad (1.35)$$

$$\ddot{z} = -\cos(\vartheta) \cos(\varphi) \frac{T}{m} + g \quad (1.36)$$

$$\ddot{\varphi} = \frac{\tau_\varphi}{I_x} \quad (1.37)$$

$$\ddot{\vartheta} = \frac{\tau_\vartheta}{I_y} \quad (1.38)$$

$$\ddot{\psi} = \frac{\tau_\psi}{I_z}. \quad (1.39)$$

Note how we changed the notation of the acceleration ($\ddot{x} = \dot{v}_x$, $\ddot{y} = \dot{v}_y$, $\ddot{z} = \dot{v}_z$).

1.2.1 Linear Control

Let's denote

$$\mathbf{p} = \begin{pmatrix} x \\ y \\ z \end{pmatrix}, \quad \mathbf{\Theta} = \begin{pmatrix} \varphi \\ \vartheta \\ \psi \end{pmatrix}, \quad \mathbf{\Gamma} = \begin{pmatrix} \tau_\varphi \\ \tau_\vartheta \\ \tau_\psi \end{pmatrix} \quad (1.40)$$

we re-write the state as

$$\boldsymbol{\xi} = \left(\mathbf{p}^T \quad \dot{\mathbf{p}}^T \quad \mathbf{\Theta}^T \quad \dot{\mathbf{\Theta}}^T \right)^T \quad (1.41)$$

with our input vector:

$$\mathbf{u} = \begin{pmatrix} T \\ \mathbf{\Gamma} \end{pmatrix}. \quad (1.42)$$

Let's separate the translational and rotational dynamics:

$$\ddot{\mathbf{p}} = g\mathbf{e}_3 - {}^I R_B(\mathbf{\Theta}) \frac{T}{m} \mathbf{e}_3 \quad (1.43)$$

$$\ddot{\mathbf{\Theta}} = \mathbf{J}^{-1} \mathbf{\Gamma}. \quad (1.44)$$

Managing the non-linear dynamics is hard for the real time controller. We apply linearization around an equilibrium point, in particular in an hovering condition. Let

$$\mathbf{p}_e = \begin{pmatrix} x_e \\ y_e \\ z_e \end{pmatrix}, \quad \mathbf{\Theta}_e = \begin{pmatrix} \varphi_e \\ \vartheta_e \\ \psi_e \end{pmatrix} \quad (1.45)$$

be our equilibrium point. We assume that our drone is perfectly horizontal, i.e. $\vartheta_e = 0$, and there are no active torques: $\mathbf{\Gamma} = \mathbf{0}$. The thrust perfectly balances the gravity:

$$T_e = mg. \quad (1.46)$$

We apply the hypothesis of small variations around $(\mathbf{p}_e \ \mathbf{\Theta}_e)$, hence

$$\sin x \approx x, \quad \cos x \approx 1 \quad (1.47)$$

and the dynamics equations becomes decoupled:

$$\ddot{x} = -g\vartheta \quad (1.48)$$

$$\ddot{y} = g\varphi \quad (1.49)$$

$$\ddot{z} = -\frac{\delta T}{m} \quad (1.50)$$

with δT being a small variation of the input thrust T .

TODO:

1.2.2 Basic Control and Planning Problems

The basic control objectives for a quadrotor are the following:

- attitude control,
- height control,
- position control,
- hovering-to-hovering motion,
- waypoint navigation,
- trajectory planning and tracking,
- obstacle avoidance.

With **attitude control**, we intend the process of determining the torques $\tau_\varphi, \tau_\theta, \tau_\psi$ needed to obtain a stable desired attitude $\varphi_d, \theta_d, \psi_d$. A trivial solution is a proportional-derivative (PD) action on the error.

Definition 1.2.2 : PD Controller for the Attitude

$$\tau_\varphi = K_{\varphi p}(\varphi_d - \varphi) + K_{\varphi d}(\dot{\varphi}_d - \dot{\varphi}) \quad (1.51)$$

$$\tau_\theta = K_{\theta p}(\theta_d - \theta) + K_{\theta d}(\dot{\theta}_d - \dot{\theta}) \quad (1.52)$$

$$\tau_\psi = K_{\psi p}(\psi_d - \psi) + K_{\psi d}(\dot{\psi}_d - \dot{\psi}). \quad (1.53)$$

For the **height control** we consider the dynamic model, in particular the Equation (1.36). We choose the value of T that linearizes the system.

Definition 1.2.3 : Height Control with FBL

To bring and keep the quadrotor to a desired height z_d we apply the feedback linearization plus a PD controller.

$$T = \frac{m}{\cos \vartheta \cos \varphi} [g - \ddot{z}_d - K_{zp}(z_d - z) - K_{zd}(\dot{z}_d - \dot{z})]. \quad (1.54)$$

Observation 1.2.1

A quadrotor is an underactuated system: it has 6 degrees of freedom in space (x, y, z and three attitude angles), but only 4 independent control inputs (the total thrust T and the three torque moments $\tau_\varphi, \tau_\theta, \tau_\psi$). There are no side motors to push the drone directly along the x axis or the y axis. To translate horizontally, the drone must first tilt (rotate) so that a thrust component T points in the desired direction.

Given the underactuated nature of the quadrotor, while we are controlling the altitude z we can't directly control x and y , hence we adopt a *hierarchical* control scheme.

1. First of all we assume that we have direct control on x and y , with two **virtual control inputs** u_x and u_y :

$$\ddot{x} = u_x \quad (1.55)$$

$$\ddot{y} = u_y. \quad (1.56)$$

2. These virtual control inputs are defined as a PD action on the error for stabilization:

$$u_x = \ddot{x}_d + K_{xp}(x_d - x) + K_{xd}(\dot{x}_d - \dot{x}) \quad (1.57)$$

$$u_y = \ddot{y}_d + K_{yp}(y_d - y) + K_{yd}(\dot{y}_d - \dot{y}). \quad (1.58)$$

3. From the dynamics in Equations (1.34) and (1.35), we denote T_x and T_y the normalized components of the thrust along the inertial axes x and y :

$$\ddot{x} = \underbrace{(-\cos \varphi \sin \vartheta \cos \psi - \sin \varphi \sin \psi)}_{T_x} \frac{T}{m} = T_x \frac{T}{m} \quad (1.59)$$

$$\ddot{y} = \underbrace{(-\cos \varphi \sin \vartheta \sin \psi + \sin \varphi \cos \psi)}_{T_y} \frac{T}{m} = T_y \frac{T}{m}. \quad (1.60)$$

4. Let's equate the real expressions to the virtual accelerations:

$$T_x \frac{T}{m} = u_x \implies T_x = \frac{m}{T} u_x = \frac{m}{T} [K_{xp}(x_d - x) + K_{xd}(\dot{x}_d - \dot{x}) + \ddot{x}_d] \quad (1.61)$$

$$T_y \frac{T}{m} = u_y \implies T_y = \frac{m}{T} u_y = \frac{m}{T} [K_{yp}(y_d - y) + K_{yd}(\dot{y}_d - \dot{y}) + \ddot{y}_d] \quad (1.62)$$

in this way we get the desired value for T_x and T_y in order to achieve u_x and u_y .

5. At this point we have to solve (1.59) and (1.60) for the roll and the pitch to get the desired values φ_d and ϑ_d :

$$\varphi_d = \arcsin (T_y \cos \psi - T_x \sin \psi) \quad (1.63)$$

$$\vartheta_d = -\arcsin \left(\frac{T_x \cos \psi + T_y \sin \psi}{\cos \varphi_d} \right). \quad (1.64)$$

6. In the end the inner loop will generate the desired values for $(\tau_\varphi, \tau_\vartheta, \tau_\psi)$ to achieve φ_d and ϑ_d .

To recap the whole control scheme:

- The quadrotor has no laterally oriented propellers, but generates a single main thrust T perpendicular to its frame. Consequently, there is no independent direct force on x and y .
- T is set as a priority to support the weight of the quadrotor and govern the z altitude.
- If we keep the drone perfectly horizontal, the thrust only points upwards. If, however, we tilt it to roll (φ) and pitch (ϑ), the thrust vector breaks down: one part continues to compensate for gravity along z , while the other is projected onto the horizontal plane.
- The position loop calculates how much horizontal acceleration $(\ddot{x}, \ddot{y})$ is needed to reach the target and, working backwards, determines exactly which roll and pitch angles (φ_d, ϑ_d) project the necessary amount of thrust T in that precise direction.

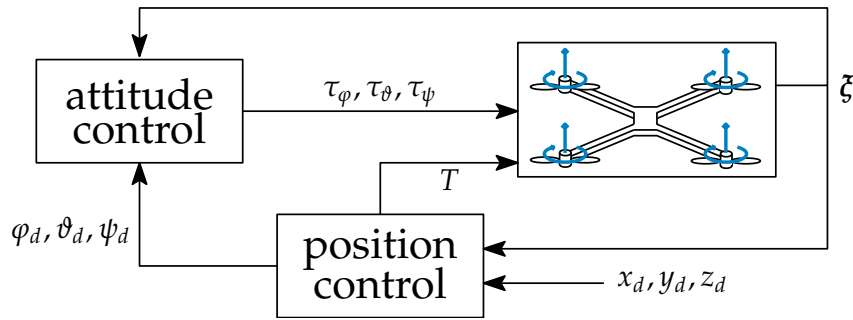


Figure 1.6: Structure of the control system.

CHAPTER

2

CONTROL METHODS

2.1 Backstepping

For a dynamic system:

$$\begin{cases} \dot{\mathbf{x}} = f(\mathbf{x}) \\ f(\mathbf{0}) = \mathbf{0} \end{cases} \quad (2.1)$$

we recall that a Lyapunov function is a scalar function that depends on $\mathbf{x}$, is continuously differentiable and positive definite:

$$V(\mathbf{x}) > 0, \quad \forall \mathbf{x} \neq \mathbf{0} \quad (2.2)$$

$$V(\mathbf{0}) = 0. \quad (2.3)$$

If $\dot{V} = \frac{\partial V}{\partial \mathbf{x}} f(\mathbf{x})$ is negative definite, then the system is asymptotically stable at the equilibrium $\mathbf{x} = \mathbf{0}$. We also know that:

- If V is radially unbounded, the system is globally asymptotically stable.
- If $\exists c_1 > 0, c_2 > 0, c_3 > 0$ with $c_1 \leq c_2$ such that

$$c_1 \|\mathbf{x}\|^2 \leq V \leq c_2 \|\mathbf{x}\|^2, \quad (2.4)$$

$$\dot{V} \leq -c_3 \|\mathbf{x}\|^2 \quad (2.5)$$

the system is globally exponentially stable.

- If $\dot{V}$ is negative semidefinite, the asymptotic stability can be proved using LaSalle's invariance principle.

In general, V is hard to find, so the backstepping method is used to construct both V and the control law, it renders Globally Asymptotically Stable (GAS) an equilibrium point or a trajectory of a nonlinear system.

The assumption is that our system is in strict-feedback (triangular) form, the quadrotor model satisfies this assumption. We will construct a simple 2-dimensional example of how backstepping works. We have the following nonlinear system:

$$\dot{x}_1 = f_1(x_1) + g_1(x_1)x_2 \quad (2.6)$$

$$\dot{x}_2 = f_2(x_1, x_2) + g_2(x_1, x_2)u \quad (2.7)$$

with $x_1 = 0$ being an equilibrium for the first subsystem. Notice the triangular structure: the first subsystem that describes the dynamics of x_1 depends nonlinearly only on x_1 , and linearly from x_2 . The second subsystem depends linearly from u (control input) and nonlinearly on (x_1, x_2) . We also assume that

$$\begin{aligned} g_1(x_1) &\neq 0 \\ g_2(x_1, x_2) &\neq 0 \end{aligned} \quad \forall (x_1, x_2). \quad (2.8)$$

- **Step 1:** We stabilize x_1 , by assuming that x_2 is a control input (can be chosen arbitrarily):

$$x_2 = \alpha(x_1) = \frac{1}{g_1(x_1)}[-f_1(x_1) - k_1 x_1] \quad (2.9)$$

with the Lyapunov function $V_1 = \frac{1}{2}x_1^2$. This leads the system to a simple differential equation:

$$\dot{x}_1 = -k_1 x_1 \quad (2.10)$$

and

$$\dot{V}_1 = -k_1 x_1^2 \quad (2.11)$$

hence the controller will globally stabilize x_1 to 0.

- **Step 2:** In reality, x_2 has its own dynamics and is not necessarily equal to our *virtual* control input $\alpha(x_1)$, hence we define an error:

$$z = x_2 - \alpha(x_1) \quad (2.12)$$

the dynamics of this error is

$$\dot{z} = \dot{x}_2 - \dot{\alpha} \quad (2.13)$$

we substitute $\dot{x}_2$ with its dynamics:

$$\dot{z} = f_2 + g_2 u - \frac{\partial \alpha}{\partial x_1} \dot{x}_1 \quad (2.14)$$

notice how we dropped the dependencies (for the sake of notation). Now we substitute $\dot{x}_1$ with its dynamics:

$$\dot{z} = f_2 + g_2 u - \frac{\partial \alpha}{\partial x_1} (f_1 + g_1 x_2). \quad (2.15)$$

For the system in Equation (2.15) we consider the following Lyapunov function:

$$V = \frac{1}{2}x_1^2 + \frac{1}{2}z^2$$

that is zero only at $x_1 = 0, z = 0$. We want to stabilize z to 0 (nullify the error on x_2). The derivative is:

$$\dot{V} = -k_1 x_1^2 + x_1 g_1 z + z(f_2 + g_2 u - \dot{\alpha}). \quad (2.16)$$

To reach the desired behavior, we choose u (our real control input) in order to make $\dot{V}$ negative definite:

$$u = \frac{1}{g_2}[-f_2 + \dot{\alpha} - k_2 z - x_1 g_1] \implies \quad (2.17)$$

$$\dot{V} = -k_1 x_1^2 - k_2 z^2 \leq -2 \min(k_1, k_2) V \quad (2.18)$$

with k_1, k_2 being two positive real numbers. The term $-k_2 z$ drives x_2 to $\alpha(x_1)$. This control law achieves exponential stability for the system in x_1 and z , and global asymptotic stability in x_1 and x_2 . The achieved closed loop system in the error coordinates is

$$\begin{pmatrix} \dot{x}_1 \\ \dot{z} \end{pmatrix} = \begin{pmatrix} -k_1 & g_1 \\ -g_1 & -k_2 \end{pmatrix} \begin{pmatrix} x_1 \\ z \end{pmatrix}. \quad (2.19)$$

The backstepping method can be recursively applied to a general triangular n -th order system:

$$\dot{x}_i = f_i(x_1, \dots, x_i) + g_i(x_1, \dots, x_i)x_{i+1}, \quad i = 1 \dots n-1 \quad (2.20)$$

$$\dot{x}_n = f_n(\mathbf{x}) + g_n(\mathbf{x})u. \quad (2.21)$$

For each step we define the error z_i , with the addition of a term on the Lyapunov function:

$$\dot{z}_i = -k_i z_i - g_{i-1} z_{i-1} + g_i z_{i+1} \quad (2.22)$$

$$\dot{V} = -\sum_i k_i z_i^2. \quad (2.23)$$

The backstepping method is a design procedure, not a defined control law, and it doesn't need to cancel the non linearities of a system to achieve stability. The problem is that the complexity of this procedure grows exponentially since each step needs the analytical derivative of the previous virtual input.

2.1.1 Backstepping for Height Control

Let's see an application of the backstepping. The height dynamics of the quadrotor is the following:

$$\dot{z} = v_z \quad (2.24)$$

$$v_z = g - \cos \varphi \cos \vartheta \frac{T}{m} \quad (2.25)$$

we choose T in order to compensate the gravity and linearize the system:

$$T = \frac{m}{\cos \varphi \cos \vartheta} (g - \ddot{z}) \implies \dot{v}_z = \ddot{z} \quad (2.26)$$

with $\ddot{z}$ to be chosen later. Given the desired height $z_d(t)$ we define the error:

$$e_z = z_d - z \implies \dot{e}_z = \dot{z}_d - v_z. \quad (2.27)$$

If the velocity v_z would have been the control input, we would have chose:

$$v_z = K_{zp} e_z + \dot{z}_d \implies \dot{e}_z = -K_{pz} e_z \quad (2.28)$$

but v_z has its own dynamics, hence we consider the formula in Equation (2.28) as a virtual control input denoted v_z^d :

$$v_z^d = K_{zp} e_z + \dot{z}_d \quad (2.29)$$

so we can define the error between the virtual input and the real velocity:

$$e_{v_z} = v_z^d - v_z \quad (2.30)$$

furthermore:

$$\dot{e}_z = -K_{pz} e_z + e_{v_z} \quad (2.31)$$

$$\dot{e}_{v_z} = K_{zp} \dot{e}_z + \ddot{z}_d - \ddot{z}. \quad (2.32)$$

The Lyapunov function becomes:

$$V = \frac{1}{2} (e_z^2 + e_{v_z}^2) \quad (2.33)$$

$$\dot{V} = -K_{pz} e_z^2 + e_{v_z} (e_z + K_{zp} \dot{e}_z + \ddot{z}_d - \ddot{z}) \quad (2.34)$$

now we choose $\ddot{z}$ to make $\dot{V}$ negative definite

$$\ddot{z} = \ddot{z}_d + e_z + K_{zp} \dot{e}_z + K_{zd} e_{v_z} \implies \quad (2.35)$$

$$\dot{V} = -K_{pz} e_z^2 - K_{dz} e_{v_z}^2 \quad (2.36)$$

by putting everything together, the final control law is:

$$T = \frac{m}{\cos \varphi \cos \vartheta} (g - \ddot{z}_d - (1 + K_{zp} K_{zd}) e_z - (K_{zp} + K_{zd}) \dot{e}_z). \quad (2.37)$$

2.1.2 PD Compared With Backstepping

What are the differences between the PD seen in Section 1.2.2 and the backstepping? They share a similar structure, but with a different choice of coordinates. The control laws are identical if we set the gain of the PD as follows:

$$K_p = 1 + K_{zp}K_{zd} \quad (2.38)$$

$$K_d = K_{zp} + K_{zd}. \quad (2.39)$$

The PD controller aim to nullify the position and velocity errors ($e_z, \dot{e}_z$) while the backstepping law instead of $\dot{e}_z$ considers $e_{v_z} = \dot{e}_z + K_{zp}e_z$, hence the desired velocity already contains a term relative to the **position correction**.

The gains have different roles:

- K_{zp} controls the decay rate of the position error, when the velocity error is zero.
- K_{zd} is the decay rate of the velocity error (the inner loop of the backstepping).
- In the PD controller, the speed and damping of the errors are given by

$$\omega_n = \sqrt{K_p} \quad (2.40)$$

$$\zeta = \frac{K_d}{2\sqrt{K_p}}. \quad (2.41)$$

With the same choices for the gains (for example, $K_p = K_d = 2$), the closed loop poles are:

- **backstepping**: $-2 \pm i$
- **PD**: $-1 \pm i$ (twice as slow).

A PD controller can be obtained from the backstepping procedure. The backstepping provides a hierarchical tuning procedure, with an explicit Lyapunov function.

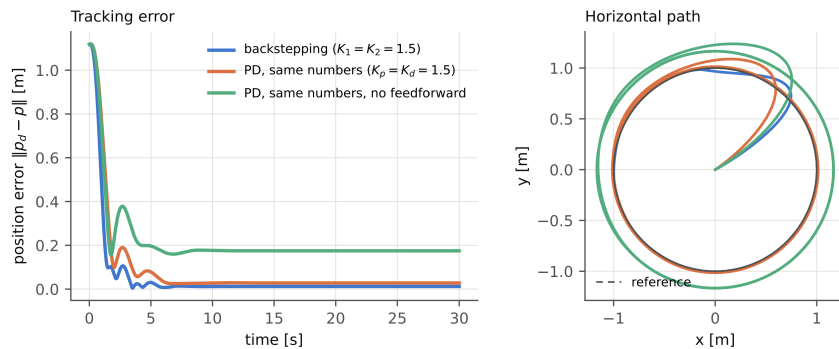


Figure 2.1: Simulated comparison between backstepping and PD.

2.1.3 Integral Backstepping

Now we consider the roll dynamics:

$$\ddot{\varphi} = \frac{\tau_{\varphi}}{I_x} + h_{\varphi} \quad (2.42)$$

h_{φ} represents the nonlinear terms, for example the gyroscopic effects. We define the integral action of the error:

TODO: finire pacco di slide 02

2.2 Dynamic Feedback Linearization

One of the most important and effective way to control a nonlinear dynamical system is by making them linear trough feedback.

Definition 2.2.1 : Feedback Linearization

The feedback linearization is the exact cancellation of the nonlinearities by feedback and change of coordinates.

Recap of the model: For control purposes, we are using the (already seen) simplified model that assumes small changes for φ and ϑ

$$(\dot{\varphi}, \dot{\vartheta}, \dot{\psi}) \simeq (p, q, r)$$

and negligible aerodynamics/gyroscopic effects:

$$\ddot{x} = -(\cos \psi \sin \vartheta \cos \varphi + \sin \psi \sin \varphi) \frac{T}{m} \quad (2.43)$$

$$\ddot{y} = -(\sin \psi \sin \vartheta \cos \varphi - \sin \varphi \cos \psi) \frac{T}{m} \quad (2.44)$$

$$\ddot{z} = -\cos \vartheta \cos \varphi \frac{T}{m} + g \quad (2.45)$$

$$\ddot{\varphi} = \frac{\tau_{\varphi}}{I_x} \quad (2.46)$$

$$\ddot{\vartheta} = \frac{\tau_{\vartheta}}{I_y} \quad (2.47)$$

$$\ddot{\psi} = \frac{\tau_{\psi}}{I_z}. \quad (2.48)$$

We have used the symbol ξ for the state and $\mathbf{u}$ for the inputs. The compact form is

$$\begin{cases} \ddot{\mathbf{p}} = g\mathbf{e}_3 - \frac{T}{m}\mathbf{b}(\Theta) \\ \ddot{\Theta} = J^{-1}\Gamma. \end{cases} \quad (2.49)$$

where we defined:

$$\mathbf{b}(\Theta) = {}^I R_B(\Theta)\mathbf{e}_3 \quad (2.50)$$

$$\mathbf{p} = (x \ y \ z)^T \quad (2.51)$$

$$\Theta = (\varphi \ \vartheta \ \psi)^T \quad (2.52)$$

$$\Gamma = (\tau_{\varphi} \ \tau_{\vartheta} \ \tau_{\psi})^T \quad (2.53)$$

$$J = \text{diag}(I_x, I_y, I_z). \quad (2.54)$$

2.2.1 Static Linearization

Before discussing the linearization of the quadrotor model, we will give some definitions for a general nonlinear system, hence keep in mind the model in Equation (2.49) for later.

Consider the following nonlinear system:

$$\dot{\xi} = f(\xi) + g(\xi)\mathbf{u} \quad \begin{matrix} \xi \in \mathbb{R}^n \\ \mathbf{u} \in \mathbb{R}^m. \end{matrix} \quad (2.55)$$

If possible, we want to find a **static** state feedback and a change of coordinates:

$$\mathbf{u} = \alpha(\xi) + \beta(\xi)\mathbf{v} \quad (2.56)$$

$$\mathbf{z} = \phi(\xi) \quad (2.57)$$

with β a non-singular function and ϕ a diffeomorphism, such that the closed loop system becomes linear and controllable:

$$\dot{\xi} = f(\xi) + g(\xi)\mathbf{u} \implies \quad (2.58)$$

$$\dot{\xi} = f(\xi) + g(\xi)(\alpha(\xi) + \beta(\xi)\mathbf{v}) \quad (2.59)$$

since $\xi = \phi^{-1}(\mathbf{z})$:

$$\dot{\phi}^{-1}(\mathbf{z}) = f(\phi^{-1}(\mathbf{z})) + g(\phi^{-1}(\mathbf{z}))(\alpha(\phi^{-1}(\mathbf{z})) + \beta(\phi^{-1}(\mathbf{z}))\mathbf{v}) \implies \quad (2.60)$$

$$\dot{\mathbf{z}} = A\mathbf{z} + B\mathbf{v} \quad (2.61)$$

with A and B being constant matrices. This linearization is exact and holds whenever β and ϕ are well defined, the control designed is completed by choosing $\mathbf{v}$ through the rules of linear control theory, and since ϕ is a diffeomorphism, by controlling $\mathbf{z}$ we can indirectly control ξ .

2.2.2 Input-Output Linearization

With this method, we focus on a subset of the state vector that we want to control, by allowing (in general) a residual internal dynamic on the unchosen variables. We choose m variables (same number of inputs) with a (sufficiently smooth) function h :

$$\boldsymbol{\eta} = h(\xi) \in \mathbb{R}^m \quad (2.62)$$

We know that the inputs $\mathbf{u}$ are present in $\dot{\xi}$, hence by differentiating $h(\xi)$, at some point one or more inputs of $\mathbf{u}$ should appear. $\boldsymbol{\eta}$ is a vector of functions η_i , $1 \leq i \leq m$, we have to differentiate each of these functions until *at least* one of the inputs of $\mathbf{u}$ appears.

Algorithm 1 Input-Output Linearization

```

1: for  $i = 1, \dots, m$  do
2:   for  $r_i = 1, \dots, +\infty$  do
3:     for  $j = 1, \dots, m$  do
4:       if  $u_j$  is present in  $\eta_i^{(r_i)}$  then
5:         save  $\eta_i^{(r_i)}$ 
6:         break
7:       end if
8:     end for
9:   end for
10: end for
```

The explicit expression of $\eta_i^{(r_i)}$ is

$$\eta_i^{(r_i)} = \underbrace{L_f^{r_i} h_i(\xi)}_{\text{Lie derivative}} + \sum_{j=1}^m L_{g_j} L_f^{r_i-1} h_i(\xi) u_j. \quad (2.63)$$

where r_i is the relative degree of differentiation for the i -th component of $\boldsymbol{\eta}$. Clearly, h should be in the class C^r with $r = \max_i r_i$. The Lie derivatives are defined as follows:

$$L_f^0 h_i(\xi) = h_i(\xi) \quad (2.64)$$

$$L_f^1 h_i(\xi) = L_f h_i(\xi) = \frac{\partial h_i(\xi)}{\partial \xi} f(\xi) \quad (2.65)$$

$$L_f^k h_i(\xi) = \frac{\partial (L_f^{k-1} h_i)}{\partial \xi} f(\xi) \quad (2.66)$$

$$(2.67)$$

$L_{g_j} L_f^{r_i-1} h_i(\xi)$ represents the Lie derivative of the intermediate scalar function $L_f^{r_i-1} h_i(\xi)$ along the input vector field $g_j(\xi)$:

$$L_{g_j} L_f^{r_i-1} h_i(\xi) = \frac{\partial(L_f^{r_i-1} h_i(\xi))}{\partial \xi} g_j(\xi). \quad (2.68)$$

About the Notation 2.2.1

When as superscript to a symbol there is a symbol between round brackets, we are denoting a derivative, if there is no brackets, it's an exponent or index:

$$a^k \text{ is } a \text{ with } k \text{ as exponent} \quad (2.69)$$

$$a^{(k)} \text{ is } a \text{ (function) differentiated up to the } k\text{-th order.} \quad (2.70)$$

For the particular case of the Lie derivatives, the superscript k in L_f^k does not represent an exponent but an order of differentiation, even if it's not in the brackets.

We can stack the m equations defined in (2.63) and write the system in compact form:

$$\begin{pmatrix} \eta_1^{(r_1)} \\ \vdots \\ \eta_m^{(r_m)} \end{pmatrix} = I(\xi) + D(\xi) \mathbf{u} \quad (2.71)$$

$D(\xi)$ is called the **Decoupling Matrix**, if it's non singular. Let $\mathbf{v} \in \mathbb{R}^m$, by the choice of the following control law:

$$\mathbf{u} = J^{-1}(\xi)(-I(\xi) + \mathbf{v}) \quad (2.72)$$

we achieve the following dynamic behavior:

$$\eta_i^{(r_i)} = v_i, \quad i = 1, \dots, m \quad (2.73)$$

we obtained m **decoupled chains of integrators** from v_i to η_i .

- if $\sum_i r_i = n$, we have **full state linearization**, and $z = (\eta_1, \eta_1^{(1)} \dots, \eta_m^{(r_m-1)})$ for $i = 1 \dots m$ are coordinates.
- if $\sum_i r_i < n$, there will be an $(n - \sum_i r_i)$ -dimensional internal dynamics, whose stability must be checked separately.

2.2.3 Linearization of the Quadrotor Model

From the quadrotor model in Equation (2.49) we consider as natural output the position and the yaw angle:

$$\boldsymbol{\eta} = (x \quad y \quad z \quad \varphi)^T \quad (2.74)$$

the model relative only to these variables is:

$$\boldsymbol{\eta} = \begin{pmatrix} \ddot{\mathbf{p}} \\ \ddot{\varphi} \end{pmatrix} = \begin{pmatrix} g \mathbf{e}_3 \\ 0 \end{pmatrix} + \begin{pmatrix} -\frac{1}{m} \mathbf{b}(\boldsymbol{\Theta}) & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ 0 & 0 & 0 & \frac{1}{I_z} \end{pmatrix} \begin{pmatrix} T \\ \tau_\varphi \\ \tau_\theta \\ \tau_\psi \end{pmatrix}. \quad (2.75)$$

The quadrotor have six degrees of freedom and 4 control inputs, its underactuated nature makes impossible to linearize the full state with a simple static feedback linearization $\mathbf{u} = \alpha(\xi) + \beta(\xi)\mathbf{v}$. If we tried to linearize only the output statically, we would be left with non-linearized internal dynamics.

To overcome this problem, we must add a **dynamic compensator**, by introducing a set of ν additional states to the controller:

$$\boldsymbol{\zeta} \in \mathbb{R}^\nu \text{ (additional states)} \quad (2.76)$$

we have to specify a dynamics for these states:

$$\dot{\boldsymbol{\zeta}} = \mathbf{a}(\boldsymbol{\zeta}, \boldsymbol{\zeta}) + \mathbf{b}(\boldsymbol{\zeta}, \boldsymbol{\zeta})\mathbf{u} \quad (2.77)$$

defined by the functions $\mathbf{a}, \mathbf{b}$ such that, in addition with the controller:

$$\mathbf{u} = \mathbf{c}(\boldsymbol{\zeta}, \boldsymbol{\zeta}) + \mathbf{d}(\boldsymbol{\zeta}, \boldsymbol{\zeta})\mathbf{v} \quad (2.78)$$

for some functions $\mathbf{c}, \mathbf{d}$ the closed loop system, of dimension $n + v$, is equivalent under a state transformation $\mathbf{z} = \boldsymbol{\phi}(\boldsymbol{\zeta}, \boldsymbol{\zeta})$ to a *linear controllable system*. Physically in the quadcopter, the auxiliary state $\boldsymbol{\zeta}$ typically consists of adding integrators on the thrust input T (thrust). By integrating the thrust once or twice, the true thrust becomes a state coordinate of the extended system, delaying the immediate appearance of the input into the derivatives and allowing the rotational dynamics to align with the translational ones without singularities.

Let's see the **algorithm of dynamic extension**: the construction procedure used to find the dynamic compensator and completely linearize the system.

1. We choose an m -dimensional output $\boldsymbol{\eta} = \mathbf{h}(\boldsymbol{\zeta})$.
2. We derive the outputs with respect to time until the inputs appear such that the decoupling matrix is non-singular (invertible).
3. When differentiating the components of $\boldsymbol{\eta}$, it often happens that some inputs appear at a lower derivation order than others, making the decoupling matrix singular. We add integrators on those input channels: the inputs become states of the compensator, their derivatives are the new input.
4. If, at the end of the algorithm, the sum of the derivation orders reached for each output equals the size of the extended state space:

$$\sum_{i=1}^m r_i = n + v$$

(where n is the size of the original physical state and v is the total number of integrators added), *full input-state-output linearization* is obtained.

5. Applying the inversion law on the extended system, the behavior between the new inputs v_i and the physical outputs η_i is exactly equivalent to m independent and decoupled chains of pure integrators:

$$\eta_i^{(r_i)} = v_i, \quad i = 1, \dots, m. \quad (2.79)$$

The linearizing quadrotor output that we choose is the position with the yaw angle:

$$\boldsymbol{\eta} = (x \quad y \quad z \quad \varphi)^T \quad (2.80)$$

The model is the one in Equation (2.75). The input appears in a non singular way only after differentiating η_1, η_2, η_3 four times and η_4 two times. The explicit expression of the vector $\mathbf{b}$ in the model is

$$\mathbf{b}(\boldsymbol{\Theta}) = {}^I R_B(\boldsymbol{\Theta}) \mathbf{e}_3 = \begin{pmatrix} \cos \psi \sin \vartheta \cos \varphi + \sin \psi \sin \varphi \\ \sin \psi \sin \vartheta \cos \varphi - \cos \psi \sin \varphi \\ \cos \vartheta \cos \varphi \end{pmatrix} \quad (2.81)$$

we will need it's Jacobian:

$$\frac{\partial \mathbf{b}}{\partial \boldsymbol{\Theta}} = \begin{pmatrix} \sin \psi \cos \varphi - \cos \psi \sin \vartheta \sin \varphi & \cos \psi \cos \vartheta \cos \varphi & \cos \psi \sin \varphi - \sin \psi \sin \vartheta \cos \varphi \\ -\cos \psi \cos \varphi - \sin \psi \sin \vartheta \sin \varphi & \sin \psi \cos \vartheta \cos \varphi & \sin \psi \sin \varphi + \cos \psi \sin \vartheta \cos \varphi \\ -\cos \vartheta \sin \varphi & -\sin \vartheta \cos \varphi & 0 \end{pmatrix}. \quad (2.82)$$

By differentiating η_1, η_2, η_3 two times we get the translational part of the model:

$$(\eta_1 \quad \eta_2 \quad \eta_3)^T = \ddot{\mathbf{p}} = g\mathbf{e}_3 - \frac{T}{m}\mathbf{b}(\boldsymbol{\Theta}) \quad (2.83)$$

we keep differentiating until the inputs $T, \boldsymbol{\Gamma}$ appears:

$$\mathbf{p}^{(3)} = -\frac{1}{m}(\dot{T}\mathbf{b} + T\mathbf{B}\dot{\boldsymbol{\Theta}}) \quad (2.84)$$

one last step:

$$\mathbf{p}^{(4)} = -\frac{1}{m}(\ddot{T}\mathbf{b} + 2\dot{T}B\dot{\Theta} + TB\ddot{\Theta}) \quad (2.85)$$

$\ddot{\Theta}$ is directly mapped to Γ through the inertia matrix:

$$\mathbf{p}^{(4)} = -\frac{1}{m}(\ddot{T}\mathbf{b} + 2\dot{T}B\dot{\Theta} + TB\mathbf{J}^{-1}\Gamma) \quad (2.86)$$

finally the inputs Γ appears in the cartesian snap $\mathbf{p}^{(4)}$. Since $\eta_4 = \varphi$, we need just two derivations in order to get a non singular dependance from τ_φ :

$$\ddot{\varphi} = \frac{\tau_\varphi}{I_x}. \quad (2.87)$$

In order to control $\mathbf{p}^{(4)}$ we have to choose $\ddot{T}$, hence this will be our input, while T and $\dot{T}$ becomes states:

$$\zeta_1 = T, \quad \zeta_2 = \dot{T} \quad (2.88)$$

the model becomes:

$$\begin{pmatrix} \mathbf{p}^{(4)} \\ \psi^{(2)} \end{pmatrix} = \underbrace{\begin{pmatrix} -\frac{1}{m}(2\zeta_2 B \dot{\Theta} + \zeta_1 \dot{B} \dot{\Theta}) \\ 0 \end{pmatrix}}_{I(\zeta, \dot{\zeta})} + \underbrace{\begin{pmatrix} -\frac{1}{m}B & -\frac{\zeta_1}{m}B\mathbf{I}^{-1} \\ 0 & 0 & 0 & \frac{1}{I_z} \end{pmatrix}}_{D(\zeta, \dot{\zeta})} \begin{pmatrix} \ddot{T} \\ \tau_\varphi \\ \tau_\theta \\ \tau_\psi \end{pmatrix} \quad (2.89)$$

In order to have the inputs appear in a non singular way, D must be non singular. Its determinant is:

$$\det D = -\frac{T^2 \cos \varphi}{m^3 I_x I_y I_z} \quad (2.90)$$

is non singular if and only if $T \neq 0$ (free fall) and $\varphi \neq \pm \frac{\pi}{2}$. Away from the singularities, the sum of the relative degrees is

$$\sum_i r_i = 4 + 4 + 4 + 2 = 14 = n + v \quad (2.91)$$

hence *full state linearization* with no zero dynamics is achieved.

TODO: slide 14